

Pollinator Habitat — System Administration Guide

This guide is written for IT staff and system administrators responsible for deploying, configuring, and maintaining the Pollinator Habitat application. It does not assume deep software development knowledge, but does assume familiarity with Linux/macOS terminals, Docker, and basic networking.

Table of Contents

- [1. What This System Does](#)
 - [2. System Requirements](#)
 - [3. Services Overview](#)
 - [4. Network & Port Map](#)
 - [5. First-Time Deployment](#)
 - [6. Environment Configuration](#)
 - [7. Starting, Stopping, and Restarting](#)
 - [8. Session Management](#)
 - [9. Logs & Health Monitoring](#)
 - [10. Database Backup and Restore](#)
 - [11. Updating the Application](#)
 - [12. Common Troubleshooting](#)
 - [13. Security Notes](#)
-

1. What This System Does

Pollinator Habitat is a **web-based nature walk game** designed for visitors at a park or nature site. Visitors use their personal phones to walk a physical route, stop at marked locations, and learn facts about pollinators.

The system has three user-facing parts:

Component	Who uses it	How they access it
Player app	Park visitors (players)	Scan a QR code → opens in phone browser automatically
Admin portal	Conner Prairie staff	Browser at the site's URL on port 3001
Backend API	The apps (not humans)	Internal — port 4000, never exposed directly

How players join: Players scan a QR code posted at the park. The code opens a URL that includes the session ID for that day — players are taken directly into the game with no manual input required.

Data collected: The system collects anonymous party size data (number of adults and children per group). No personal names, email addresses, or account credentials are collected from players.

2. System Requirements

Server (production):

- Linux (Ubuntu 22.04 LTS or Debian 12 recommended)
- Docker Engine 24+ and Docker Compose v2+
- 2 GB RAM minimum, 4 GB recommended

- 10 GB free disk space
- Ports 80 and 3001 open in firewall

Client devices (players):

- Any modern smartphone or tablet with a camera and browser (Chrome, Safari, Firefox)
- Camera used to scan the event QR code — both iPhone and Android recognize QR codes natively without a separate app
- Connected to the same network as the server, or server is internet-accessible
- No app installation required

Admin portal clients:

- Any modern desktop browser
- Network access to the server on port 3001

3. Services Overview

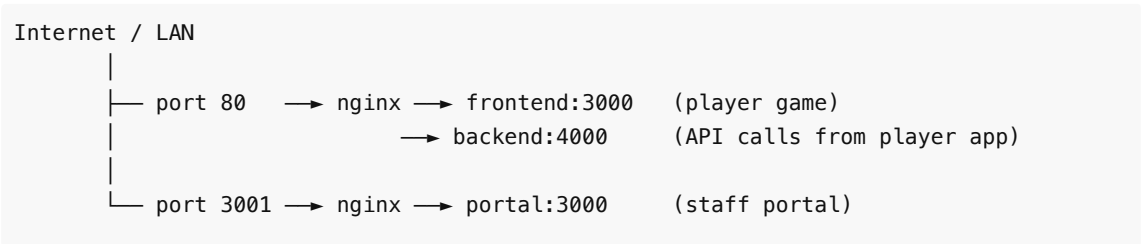
The application runs as **six Docker containers** coordinated by Docker Compose. Each container has a single responsibility:

Service	Container name	Purpose
postgres	pollinator-postgres	PostgreSQL 18 database — stores all game and survey data
migrate	pollinator-migrate	One-shot container that runs database migrations and seeds data. Exits after completion
backend	pollinator-backend	Node.js/Express API server — game logic, data access
frontend	pollinator-frontend	Next.js app — the player-facing game UI
portal	pollinator-portal	Next.js app — the staff admin portal
nginx	pollinator-nginx	Reverse proxy — routes incoming HTTP traffic to the right service

Startup order: Postgres → migrate → backend → frontend + portal → nginx.

If any service in the chain fails, the dependent services will not start. For example, if the database fails its health check, neither `migrate` nor `backend` will start.

4. Network & Port Map



→ backend:4000 (API calls from portal)

Internal only (not reachable from outside Docker):

```
backend:4000
frontend:3000
portal:3000
postgres:5432
```

Only ports 80 and 3001 are exposed to the host machine. The database, backend, and Next.js servers are not directly reachable from outside the Docker network — all traffic goes through Nginx.

URL routing (Nginx):

- `http://your-server/` → player game
- `http://your-server/api/...` → backend API (proxied transparently)
- `http://your-server:3001/` → staff portal
- `http://your-server:3001/api/...` → backend API (same backend, different entry point)

5. First-Time Deployment

Step 1 — Install Docker

```
# Ubuntu/Debian
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
# Log out and back in for group membership to take effect
```

Step 2 — Clone the repository

```
git clone <repository-url>
cd Pollinator-Habitat
```

Step 3 — Create the environment file

```
cp .env.pg.example .env.pg
nano .env.pg # or use any text editor
```

Change these values before deploying:

```
POSTGRES_PASSWORD=<choose a strong password>
DATABASE_URL=postgresql://pollinator:<your-password>@postgres:5432/pollinator
JWT_SECRET=<generate with: openssl rand -hex 32>
```

The `POSTGRES_USER`, `POSTGRES_DB`, and container hostnames should not be changed unless you also update the `DATABASE_URL` to match.

Do not deploy with the example `JWT_SECRET`. It is a known public value. Anyone with that key can forge authentication tokens.

Step 4 — Update the compose file to use your credentials file

The production compose file currently points at `.env.pg.example`. Change it to point at your `.env.pg` file. Edit `docker-compose.prod.pg.yml` and change all `env_file: - .env.pg.example` lines to `env_file: - .env.pg`.

Step 5 — Start the stack

```
docker compose -f docker-compose.prod.pg.yml up -d
```

The `-d` flag runs everything in the background. The first run takes several minutes because Docker must:

1. Pull the base images (postgres, nginx)
2. Build the application images (backend, frontend, portal)
3. Run database migrations and seed data

Step 6 — Verify everything is running

```
docker compose -f docker-compose.prod.pg.yml ps
```

All services should show `running` or `healthy`. The `migrate` service will show `exited (0)` — this is correct; it exits after completing successfully.

```
# Test the player app
curl http://localhost/

# Test the backend health check
curl http://localhost/api/health
# Should return: {"status":"ok"}

# Test the portal
curl http://localhost:3001/
```

6. Environment Configuration

All configuration is done through the `.env.pg` file. **Never commit this file to version control.**

Variable	Required	Description
POSTGRES_DB	Yes	Database name. Default: pollinator
POSTGRES_USER	Yes	Database user. Default: pollinator
POSTGRES_PASSWORD	Yes	Database password. Change from default.
DATABASE_URL	Yes	Full Prisma connection string. Must match the three values above
JWT_SECRET	Yes	Secret used to sign authentication tokens. Generate a new one.

Generate a secure `JWT_SECRET` :

```
openssl rand -hex 32
```

DATABASE_URL format:

```
postgresql://<POSTGRES_USER>:<POSTGRES_PASSWORD>@postgres:5432/<POSTGRES_DB>
```

The hostname `postgres` is the Docker service name — do not change it unless you rename the postgres service in the compose file.

Configuration that is not in `.env.pg`

The following are hardcoded in the compose file or Nginx config and would require editing those files to change:

Setting	Current value	Where to change
Player app port	80	<code>docker-compose.prod.pg.yml</code> → nginx ports
Portal port	3001	<code>docker-compose.prod.pg.yml</code> → nginx ports
Nginx config	See <code>nginx/nginx.conf</code>	Edit file, then <code>docker compose restart nginx</code>
Max DB connections	500	<code>docker-compose.prod.pg.yml</code> → postgres command

7. Starting, Stopping, and Restarting

All commands run from the `Pollinator-Habitat/` directory.

Start the stack

```
docker compose -f docker-compose.prod.pg.yml up -d
```

Stop the stack (preserves data)

```
docker compose -f docker-compose.prod.pg.yml down
```

Restart a single service

```
docker compose -f docker-compose.prod.pg.yml restart backend
docker compose -f docker-compose.prod.pg.yml restart nginx
```

Stop and remove everything including data volumes

```
# WARNING: This deletes all database data permanently
docker compose -f docker-compose.prod.pg.yml down -v
```

Check what is running

```
docker compose -f docker-compose.prod.pg.yml ps
```

Restart the entire stack without losing data

```
docker compose -f docker-compose.prod.pg.yml down
docker compose -f docker-compose.prod.pg.yml up -d
```

8. Session Management

What is a session?

A **session** represents one event day at the park. Each session has a unique 9-digit numeric ID. All game data (route completions, party size surveys) is associated with the session ID for that day.

How players join — QR codes

Players join by scanning a QR code posted at the park. The QR code encodes a direct URL to the game with the session ID embedded as a query parameter:

```
http://your-server/?sessionId=202604150
```

When a player scans the code and opens the link, the app automatically validates the session and takes them directly to the home screen — no manual code entry required.

Your responsibility as IT: Generate and provide QR codes to event staff before each event season (or per-event as needed). QR codes can be created with any standard QR code generator — simply encode the session URL for each date.

Example for April 15, 2026:

```
URL to encode: http://your-server/?sessionId=202604150
```

A free tool like `qr-code-generator.com` or the `qrencode` command-line tool can produce print-ready codes:

```
qrencode -o session_20260415.png "http://your-server/?sessionId=202604150"
```

Provide the QR code image to staff as a print file (PDF or high-res PNG, minimum 300 DPI for signage).

Fallback — manual code entry

If a visitor cannot scan the QR code, they can open the game URL directly in their browser:

```
http://your-server/
```

They will see a text field where they can type the 9-digit session code. Provide the plain URL and the session code to event staff as a backup.

Pre-seeded sessions

The database is seeded at first startup with session IDs pre-generated for every day from January 2024 through December 2024. These IDs follow the pattern `YYYYMMDD0` — for example, January 15, 2024 has session ID `202401150`.

If your events fall within this range, sessions already exist and no additional setup is needed beyond generating QR codes.

Checking whether a session ID exists

```
curl -X POST http://your-server/api/ \
  -H "Content-Type: application/json" \
  -d '{"sessionId": "202401150"}'
# Returns a JWT token if the session exists, or an error if not
```

Creating a new session (outside the pre-seeded range)

If you need a session ID not covered by the seed data (e.g. dates after December 2024):

```
curl -X POST http://your-server/api/create-session \
  -H "Content-Type: application/json" \
  -d '{"sessionId": 202512250}'
# Returns: {"sessionId": 202512250}
```

Session IDs must be numeric. There is no UI — it requires a direct API call. After creating the session, generate the QR code for that ID and deliver it to event staff.

Searching session data in the portal

After an event, staff use the portal at `http://your-server:3001` to search party size data by session ID or date range. See the [Client Guide](#) for portal usage instructions.

9. Logs & Health Monitoring

View logs for all services

```
docker compose -f docker-compose.prod.pg.yml logs
```

Follow logs in real time

```
docker compose -f docker-compose.prod.pg.yml logs -f
```

View logs for a specific service

```
docker compose -f docker-compose.prod.pg.yml logs -f backend
docker compose -f docker-compose.prod.pg.yml logs -f nginx
```

```
docker compose -f docker-compose.prod.pg.yml logs -f postgres
```

View only recent logs

```
docker compose -f docker-compose.prod.pg.yml logs --tail=100 backend
```

Health check status

Docker tracks each service's health check result:

```
docker inspect --format='{{.State.Health.Status}}' pollinator-backend
docker inspect --format='{{.State.Health.Status}}' pollinator-frontend
```

Valid statuses: `healthy` , `unhealthy` , `starting` .

Manual health check

```
curl http://localhost/api/health
# Expected: {"status":"ok"}
```

If this returns an error or times out, the backend is not running or Nginx cannot reach it.

What to check when something is wrong

1. `docker compose -f docker-compose.prod.pg.yml ps` — check which services are running
2. `docker compose -f docker-compose.prod.pg.yml logs migrate` — was the migration successful?
3. `docker compose -f docker-compose.prod.pg.yml logs backend` — is the backend crashing?
4. `docker compose -f docker-compose.prod.pg.yml logs postgres` — is the database accepting connections?

10. Database Backup and Restore

Backup

```
# Dump the entire database to a .sql file
docker exec pollinator-postgres \
  pg_dump -U pollinator pollinator > backup_$(date +%Y%m%d).sql
```

This creates a file like `backup_20260401.sql` in the current directory. Store this file somewhere safe off-server.

Restore from backup

```
# Stop dependent services first
docker compose -f docker-compose.prod.pg.yml stop backend frontend portal nginx
```



```
# Restore the dump
docker exec -i pollinator-postgres \
  psql -U pollinator pollinator < backup_20260401.sql

# Restart
docker compose -f docker-compose.prod.pg.yml start backend frontend portal nginx
```

Schedule regular backups (example cron job)

```
# Add to crontab: crontab -e
# Back up every night at 2 AM, keep 30 days
0 2 * * * cd /path/to/Pollinator-Habitat && \
  docker exec pollinator-postgres pg_dump -U pollinator pollinator > \
  /backups/pollinator_$(date +%Y%m%d).sql && \
  find /backups -name "pollinator_*.sql" -mtime +30 -delete
```

11. Updating the Application

When new code is deployed, the application images must be rebuilt.

Full update (rebuild all images)

```
cd Pollinator-Habitat/
git pull
docker compose -f docker-compose.prod.pg.yml down
docker compose -f docker-compose.prod.pg.yml build --no-cache
docker compose -f docker-compose.prod.pg.yml up -d
```

Update a single service without downtime (backend example)

```
docker compose -f docker-compose.prod.pg.yml build backend
docker compose -f docker-compose.prod.pg.yml up -d --no-deps backend
```

Applying new database migrations

If the update includes database schema changes, the `migrate` container must run again:

```
# Bring down the stack
docker compose -f docker-compose.prod.pg.yml down

# Rebuild and bring up – migrate runs automatically on startup
docker compose -f docker-compose.prod.pg.yml build
docker compose -f docker-compose.prod.pg.yml up -d
```

The `migrate` service runs `prisma migrate deploy` on every startup. It only applies migrations that have not already been applied — re-running it on an up-to-date database is safe.

12. Common Troubleshooting

QR code scans but the game shows an error

1. Verify the session ID in the QR code URL exists in the database:

```
curl -X POST http://your-server/api/ \
-H "Content-Type: application/json" \
-d '{"sessionId": "202604150"}'
```

If you get an error, the session is missing — create it with `POST /api/create-session` (see §8)

2. Check the QR code encodes the correct URL format: `http://your-server/?sessionId=XXXXXXXX`
3. Check backend logs: `docker compose -f docker-compose.prod.pg.yml logs backend`

Players cannot reach the game at all (QR code link fails to open)

1. Check that the stack is running: `docker compose -f docker-compose.prod.pg.yml ps`
2. Check Nginx is up and the frontend is healthy: `curl http://localhost/`
3. Check firewall — port 80 must be open
4. Check Nginx logs: `docker compose -f docker-compose.prod.pg.yml logs nginx`
5. Confirm the QR code encodes the correct server address (the one players can reach — not `localhost`)

Staff cannot access the portal

1. Check that port 3001 is open in the firewall
2. Check portal container is healthy: `docker inspect --format='{{.State.Health.Status}}' pollinator-portal`
3. Try: `curl http://localhost:3001/`

The portal shows "Search failed" for all searches

1. The backend may be down. Check: `curl http://localhost/api/health`
2. Check that `NEXT_PUBLIC_API_URL` is set correctly. In production it should be empty (`""`) so Nginx proxies `/api/...` correctly. If it is set to a wrong URL the portal will call a non-existent address.

Database won't start

1. Check disk space: `df -h` — PostgreSQL will refuse to start if disk is full
2. Check logs: `docker compose -f docker-compose.prod.pg.yml logs postgres`
3. If data volume is corrupted: restore from backup (see §10)

migrate exits with error code

1. Check logs: `docker compose -f docker-compose.prod.pg.yml logs migrate`
2. Common cause: database wasn't ready (health check passed too early). Re-run:

```
docker compose -f docker-compose.prod.pg.yml run --rm migrate
```

3. If a migration is failing: check backend logs for the specific SQL error. May require developer intervention.

High memory or CPU on the database container

PostgreSQL is configured with `max_connections=500` . For most event-day loads this is more than sufficient. If you observe high resource use:

- 1. Check active connections:

```
docker exec pollinator-postgres psql -U pollinator -c "SELECT count(*) FROM pg_stat_activity;"
```

- 2. If count is unexpectedly high, restart the backend: `docker compose -f docker-compose.prod.pg.yml restart backend`

13. Security Notes

Item	Recommendation
JWT_SECRET	Generate with <code>openssl rand -hex 32</code> . Rotate if ever exposed. Requires backend restart to take effect.
Database password	Use a strong random password. The DB is not exposed outside Docker, but defense-in-depth applies.
Port 3001 (portal)	Consider restricting to staff IP ranges at the firewall level. The portal has no authentication.
HTTPS	This guide covers HTTP only. For internet-facing deployments, place a TLS terminator (e.g. Nginx with Let's Encrypt, or Cloudflare) in front of port 80 and 3001.
Container runtime	All application containers run as non-root user 65532. The database container runs as the postgres user.
.env.pg file	Set file permissions to restrict read access: <code>chmod 600 .env.pg</code>
Backups	Store database backups off-server (e.g. S3, external drive). Backups on the same server are lost if the server fails.

This document covers the PostgreSQL production stack (`docker-compose.prod.pg.yml`). For MySQL deployment or development setup, see [Developer/DOCKER_SETUP.md](#).